

# Futurity247 — Project Context for Claude

This file is read by Claude (via Cursor, Claude Code, or any AI coding tool) at the start of every session. It contains the ground truth about what we're building, how, and why. Keep it up to date.

---

## 1. What this is

**Futurity247** is a vertical SaaS AI receptionist for **electrical contractors**. Each client is a small-to-midsize electrical business (solo operators up to ~10 trucks). We answer their phones 24/7 using a voice AI named **Billie**, book jobs into their calendar, escalate emergencies to the owner, and flag high-value leads in real time.

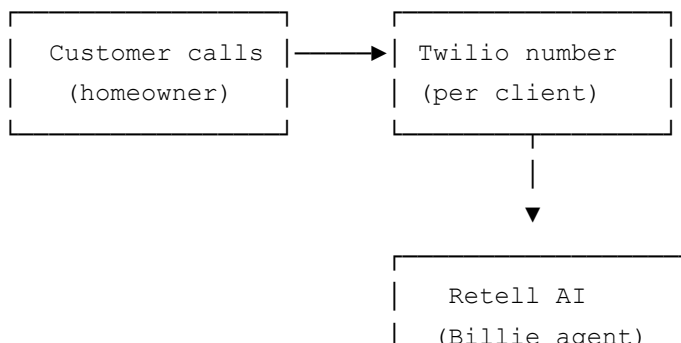
**The business model:** Monthly platform fee (\$299/\$499/\$899 tiers) + per-minute overage. 70%+ gross margin at scale.

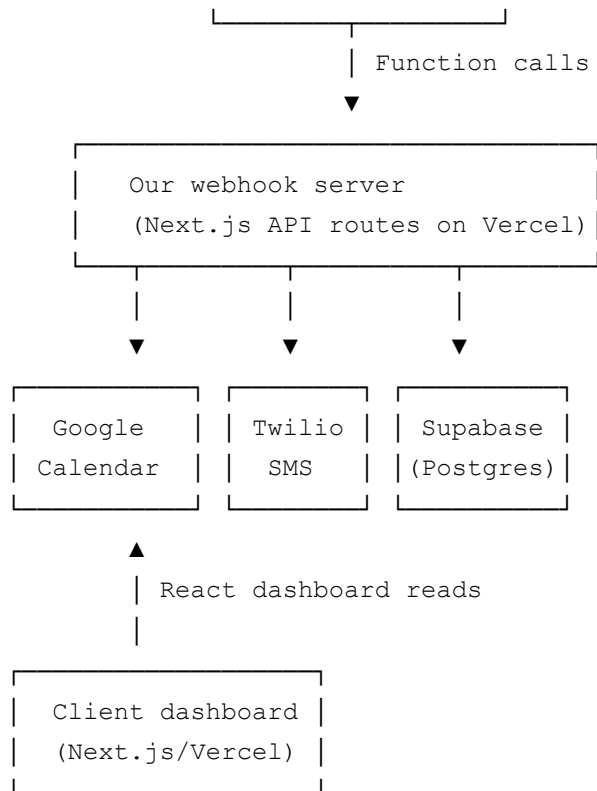
**The moat:** Vertical specialization. Every integration, prompt, emergency keyword, job-duration estimate, and lead-scoring rule is tuned for electricians — not generic receptionist software. A customer cannot replace us with Pulsy or a Retell reseller without losing this specificity.

**The 3-year goal:** Reach \$100K+ MRR in the electrician vertical, then sell the business or use the cash to launch sister brands for plumbers / HVAC / locksmiths. Valuation improves dramatically if we own the stack (Retell infrastructure + proprietary integrations + client data) rather than reselling someone else's platform.

---

## 2. Architecture





---

### 3. Tech stack (non-negotiable)

- **Frontend:** Next.js 14+ (App Router), React 18, TypeScript
- **Styling:** Tailwind CSS. No CSS modules, no styled-components.
- **UI components:** shadcn/ui where reasonable, hand-rolled for bespoke components (dashboard widgets, charts)
- **Charts:** Recharts
- **Icons:** lucide-react
- **Fonts:** Instrument Serif (display) + Geist (body). Exactly two fonts, no exceptions.
- **Database:** Supabase (Postgres + Auth + Row Level Security)
- **Auth:** Supabase Auth with magic links + Google OAuth for clients
- **Backend:** Next.js API routes for the webhook server. No separate Express server — we keep it monorepo.
- **Voice AI:** Retell AI (agent config in `/config/retell_agent_config.json`)
- **LLM in the voice agent:** Claude Sonnet 4.5 (not GPT — Claude follows our prompt more

reliably for slot-filling and emergency triage)

- **Telephony:** Twilio (numbers, SMS)
  - **Calendar:** Google Calendar via service account (each client shares their calendar with us)
  - **Hosting:** Vercel (frontend + API routes), Supabase (DB), Railway only if we need a long-running process later
  - **Payments:** Stripe (subscriptions + usage-based metering for overage minutes)
- 

## 4. Design system

### Aesthetic

Dark techno-industrial. Deep navy (#0a0e14) background. Electric cyan (#22d3ee) and lime (#a3e635) accents. Glassmorphism cards. The vibe is “premium AI product,” not “corporate SaaS.” Think Linear × Vercel × Stripe.

### Color tokens

```
bg-base:      #0a0e14    (page background)
bg-card:      rgba(18, 24, 33, 0.6)    (glass)
border:       rgba(255,255,255,0.06)
text-primary: #e2e8f0    (slate-200)
text-muted:   #94a3b8    (slate-400)
accent-cyan:  #22d3ee
accent-lime:  #a3e635
accent-warn:  #fb923c    (orange-400, for emergencies/hot leads)
accent-good:  #34d399    (emerald-400, for success states)
```

### Typography rules

- **Display** (big numbers, hero headings): `font-display` → Instrument Serif, light weight, tight tracking
- **Body & UI:** Geist, 400-600 weight
- **Small caps / labels:** Geist with `letter-spacing: 0.05em` and uppercase — we call this `font-mono-alt`. Do NOT use a monospace font.

### Motion rules

- Entrance animations: stagger fade-in over 0.3s on page load, use `fade-in-1` through `fade-in-4` classes
- Hover states on interactive elements: border color shift to cyan, 150ms ease
- No bouncy springs, no page transitions, no scroll-triggered reveals. Restraint is the point.
- Mouse-following spotlight on desktop, scroll-following on mobile (already implemented in `/components/ambient-background.tsx` )

## Do NOT

- Use purple/violet gradients (AI cliché)
  - Use Inter, Roboto, or any generic sans-serif
  - Add decorative emojis or icons where text would do
  - Use more than 2 fonts total
  - Use bright white (`#ffffff`) for body text — always slate-200 or dimmer
- 

## 5. Database schema

See `/db/schema.sql` for the full schema. Key tables:

- `clients` — one row per electrician business. Multi-tenant root. Stores Retell agent ID, Twilio number, calendar ID, pricing plan, business hours.
- `leads` — every qualified caller. Customer contact info, job description, job type, urgency, estimated value, booking status.
- `calls` — every inbound call, regardless of outcome. Transcript, summary, sentiment, outcome, duration. Used for billing + analytics.
- `monthly_usage` (view) — rollup for billing calculations.
- `client_roi` (view) — **the single most important query in the product**. Computes pipeline value generated per client per month divided by their monthly fee. This is what we show them to prevent churn.

All tables use RLS (Row Level Security) keyed to `client_id`. A client's dashboard can only ever read their own data. Service role bypasses RLS for webhook operations.

---

## 6. Voice agent conventions

The Retell agent is named **Billie**. Female voice, tuned for warmth + efficiency. Her job is NOT to be chatty or clever — her job is to get callers off the phone fast with the right outcome (booked, escalated, or messaged).

### Hard rules Billie follows (enforced in the prompt):

1. **Never quotes prices.** Always says “the electrician will give you a quote on-site.” Our service call fee is the only number she’ll name.
2. **Never diagnoses.** If asked “what’s wrong with my wiring,” she says “I don’t want to guess — that’s what the electrician is for.”
3. **Emergency keywords trigger immediate escalation.** The list is in `/lib/emergency-classifier.ts`. Any match → call `triage_emergency` function → text owner → do not try to book.
4. **Confirms phone numbers by reading them back.** Hallucinated digits = missed customer.
5. **Short responses.** One or two sentences per turn. No monologues.

### Functions Billie can call (implemented as Next.js API routes):

- `POST /api/functions/check_availability` — queries Google Calendar for open slots given a job type and preferred date
- `POST /api/functions/book_appointment` — creates calendar event, inserts lead, sends SMS confirmation, alerts owner if urgent
- `POST /api/functions/triage_emergency` — alerts owner immediately, logs the call, returns safety instructions
- `POST /api/functions/qualify_lead` — scores the lead (hot/warm/cold), flags hot leads to owner

Post-call webhook: `POST /api/webhooks/call_ended` — receives transcript + analysis from Retell, writes to `calls` table.

---

## 7. Pricing tiers (hardcoded in pricing logic)

| Plan | Monthly | Included min | Overage | Target |
|------|---------|--------------|---------|--------|
|      |         |              |         |        |

|         |       |      |            |                       |
|---------|-------|------|------------|-----------------------|
| Starter | \$299 | 300  | \$0.30/min | Solo electrician      |
| Pro     | \$499 | 500  | \$0.25/min | 2-5 trucks            |
| Premium | \$899 | 1500 | \$0.20/min | 5+ trucks, commercial |

Our cost per minute: ~\$0.08-0.12 (Retell + Twilio + LLM tokens). Never expose this internally or externally.

---

## 8. File conventions

```

/app
  / (marketing)      Public pages (landing, pricing, about)
  / (dashboard)      Authenticated client dashboard
    /calls           Call history + transcripts
    /leads           Lead pipeline
    /settings        Business config
  /api
    /functions       Retell function webhooks
    /webhooks        Inbound webhooks (Retell, Stripe, Twilio)
    /stripe          Billing endpoints
  /components
    /ui              shadcn primitives
    /dashboard       Custom dashboard widgets (StatCard, ROICard, CallRow, et
    /ambient-background.tsx The spotlight + grid background
  /lib
    /supabase        DB client + query helpers
    /retell          Agent config + function handlers
    /twilio          SMS helpers
    /calendar        Google Calendar helpers
    /emergency-classifier.ts Keyword → urgency mapping
    /pricing.ts       Plan definitions, job value estimates
  /db
    /schema.sql      Full Postgres schema
    /migrations       Future migrations
  /config
    /retell_agent_config.json Billie's prompt + function definitions

```

### Naming rules

- **Components:** PascalCase.tsx
- **Utilities:** kebab-case.ts

- API routes: follow Next.js conventions ( `route.ts` inside folders)
  - Database columns: `snake_case`
  - TypeScript types: `PascalCase` , suffixed with intent ( `Client` , `ClientInsert` , `ClientUpdate` )
- 

## 9. Coding conventions

### TypeScript

- Strict mode on. No `any` without a comment explaining why.
- Zod for runtime validation of all API route inputs.
- Types generated from Supabase schema using `supabase gen types typescript` .

### React

- Server Components by default. Add `"use client"` only when necessary (interactivity, hooks, browser APIs).
- Data fetching in Server Components with direct Supabase calls. Don't build an unnecessary API layer.
- Forms: React Hook Form + Zod.

### API routes

- Every route validates input with Zod.
- Every route returns a typed response.
- Errors are logged to console + returned as `{ error: string }` with appropriate HTTP status.
- Retell webhooks must verify the signature header before processing.

### General

- No comments explaining WHAT the code does. Comments explain WHY when the reasoning isn't obvious.
- Prefer composition over abstraction. Duplication is cheaper than the wrong abstraction.
- If a function is longer than 50 lines, consider splitting it.

---

## 10. What to build next (current priorities)

1. ☒ Dashboard UI (done — see `/components/dashboard/*` )
  2. ☒ Database schema (done — see `/db/schema.sql` )
  3. ☒ Retell agent config + function definitions (done — see `/config/retell_agent_config.json` )
  4. ☐ Wire dashboard to real Supabase data (replace mock constants)
  5. ☐ Supabase Auth integration with magic link flow
  6. ☐ Client onboarding wizard (4 steps: business info, calendar connect, Twilio number provision, phone-forwarding instructions)
  7. ☐ Stripe subscription + usage-based metering
  8. ☐ Marketing landing page + pricing page
  9. ☐ Call detail view (transcript, audio playback, outcome override)
  10. ☐ Admin panel (for us, to manage all clients)
- 

## 11. Things to never do

- **Never store credit card numbers, SSNs, or any PII beyond what's needed for the service.** Stripe handles cards. We store names, phone numbers, addresses, call transcripts. That's it.
- **Never let the voice agent quote prices, diagnose problems, or promise specific technicians by name.** This is a liability issue for the electrician.
- **Never mark a call "booked" without a corresponding Google Calendar event existing.** The calendar is the source of truth.
- **Never let one client see another client's data.** RLS is non-negotiable. Every query must filter by `client_id` at the database level, not just in application code.
- **Never build on top of Pulsy, Trillet, or other all-in-one voice platforms.** We own our stack. That's the whole point of the business.
- **Never silently retry failed SMS/calendar operations without logging.** Billing disputes come from missing confirmations.



- **Never use a font outside Instrument Serif + Geist.** Consistency is the brand.
- 

## 12. Useful reference

- Retell docs: <https://docs.retellai.com>
  - Supabase + Next.js: <https://supabase.com/docs/guides/auth/server-side/nextjs>
  - shadcn/ui: <https://ui.shadcn.com>
  - Twilio Node SDK: <https://www.twilio.com/docs/libraries/node>
  - Google Calendar API: <https://developers.google.com/calendar/api/v3/reference>
- 

**Claude, when in doubt, ask before implementing. This is a production SaaS that real businesses will depend on to answer real customer calls. Correctness > speed.**